



Institut National des Postes et Télécommunications

Projet RAG

Analyse des Choix Technologiques : Bot RAG

Élaboré par :
ESSABRI YOUNESS & AMTOUT Walid

Année Universitaire 2025-2026

Table des matières

1	Introduction	2
2	Analyse des Choix Technologiques	2
2.1	1. Extracteur	2
2.2	2. VLM pour Chunking	2
2.3	3. Embedding Model	3
2.4	4. Vector DB	3
2.5	5. LLM	3
2.6	6. Inférence	4
2.7	7. Framework	4
3	Conclusion	5

1 Introduction

Dans le cadre de notre projet de développement d'un système basé sur la Génération Augmentée par la Recherche (RAG - *Retrieval-Augmented Generation*), il est primordial de concevoir une architecture robuste et optimisée. Ce rapport détaille l'analyse des choix technologiques pour les sept composantes d'un Bot RAG moderne, en présentant pour chaque composant son utilité au sein du système RAG et les alternatives les plus pertinentes du marché.

2 Analyse des Choix Technologiques

2.1 1. Extracteur

L'outil qui nettoie nos PDF/Markdown.

Rôle du composant : Les documents au sein d'une entreprise ou institution existent sous diverses formes dites "non-structurées" (mises en page de PDF, présentations PowerPoint, fichiers Word). L'extracteur agit comme un parseur spécialisé : il ingère ces documents bruts de leurs fichiers originaux, supprime la pagination ou les pieds de page inutiles, et rassemble le texte propre.

- **Unstructured.io** : Le couteau suisse capable de tout lire.
- **LlamaParse** : Optimisé par l'IA pour les documents complexes (tableaux).
- **PyMuPDF (fitz)** : Le plus rapide pour le texte brut.
- **Docling (IBM)** : Nouveau favori pour extraire la hiérarchie des documents.
- **Marker** : Transforme les PDF en Markdown parfait (idéal pour le RAG).

Choix retenu : PyMuPDF

2.2 2. VLM pour Chunking

Modèles Vision-Langage pour découper les documents selon leur mise en page.

Rôle du composant : Pendant qu'un extracteur de texte classique occulte le sens visuel (l'emplacement d'un titre, la subtilité d'un graphique financier ou la structuration exacte d'un tableau administratif), le *Vision-Language Model* (VLM) vient combler cette lacune. Son rôle novateur est d'interpréter le document source comme une "image" ou un plan visuel d'abord, ce qui permet un découpage (chunking) du contexte documentaire beaucoup plus pertinent pour les cas d'utilisation hautement complexes qui reposeraient inévitablement sur un support visuel fort.

- **ColPali** : Un modèle révolutionnaire qui "voit" les pages au lieu de les lire.
- **GPT-4o / Gemini 1.5 Pro** : Capables d'analyser visuellement un PDF entier.
- **Qwen2-VL** : Très performant en open-source pour comprendre les schémas.
- **LayoutLMv3** : Spécialisé dans la compréhension de la structure des formulaires.
- **GLM-4V** : Une alternative puissante pour l'analyse visuelle de documents.

Choix retenu : GPT-4o

2.3 3. Embedding Model

L'outil qui transforme le texte en coordonnées mathématiques.

Rôle du composant : Le modèle d'embedding (plongement de mots) est le pont linguistique fondamental de l'application : la machine ne comprend pas les mots, elle ne traite que des chiffres. Ce modèle spécifique (bien plus petit et rapide que le LLM) convertit les phrases de nos documents et la question de notre utilisateur en listes denses de centaines ou milliers de nombres flottants. Ce procédé permet de capturer sémantiquement des relations comme la synonymie (par exemple, localiser le mot "navire" même si l'utilisateur a tapé "bateau").

- **OpenAI text-embedding-3-large** : Le choix sûr et performant.
- **BGE-M3 (BAAI)** : Le roi de l'open-source, multilingue et très précis.
- **Nomic Embed Text** : Excellent pour le local et les grandes fenêtres de contexte.
- **Voyage-3.5** : Spécialisé pour les documents techniques et le code.
- **Cohere Embed v4** : Le meilleur pour le multilingue en entreprise.

Choix retenu : OpenAI text-embedding-3-large

2.4 4. Vector DB

Où sont stockés nos documents transformés en vecteurs.

Rôle du composant : La base de données vectorielle est l'espace de stockage ultra-rapide conçu spécialement pour héberger et manipuler d'énormes collections de suites de chiffres (embeddings). Contrairement à une base de données MySQL classique, elle ne recherche pas le mot-clé exact tapé par l'utilisateur, mais effectue instantanément des calculs géométriques complexes (comme la similarité cosinus) pour trouver le texte dont "le sens global" est mathématiquement le plus proche de la question posée. C'est elle qui permet de récupérer rapidement le contexte pertinent.

- **Qdrant** : Ultra-rapide (Rust) avec des filtres avancés.
- **Pinecone** : Le plus simple, 100% géré dans le cloud (Serverless).
- **ChromaDB** : Le meilleur pour débiter localement (`pip install`).
- **Milvus** : Conçu pour les échelles massives (milliards de vecteurs).
- **Weaviate** : Excellent pour combiner recherche vectorielle et mots-clés (Hybrid Search).
- **pgvector (Postgres)** : Si vous voulez tout garder dans votre base de données SQL.

Choix retenu : ChromaDB

2.5 5. LLM

Le modèle qui raisonne et rédige.

Rôle du composant : Le *Large Language Model* (LLM) constitue l'intelligence artificielle proprement dite, le cœur logique du traitement. Son rôle dans l'architecture RAG est de lire le "contexte" (les informations issues des documents de l'entreprise) récupéré par le moteur de recherche, de le synthétiser, le comprendre mathématiquement, puis de formuler une réponse naturelle, cohérente et idiomatique répondant exactement à la question de l'utilisateur.

- **Llama 3.3 (ou 4 Maverick)** : Le standard open-weight polyvalent.
- **DeepSeek-V3 / R1** : Les nouveaux champions du rapport performance/prix et du raisonnement.
- **Claude 3.5 Sonnet** : Souvent considéré comme le plus "intelligent" pour le code et les nuances.
- **Qwen 3.5 (Alibaba)** : Exceptionnel pour le multilingue et les contextes très longs.
- **Mistral Large** : La référence européenne, très robuste pour les entreprises.

Choix retenu : OpenAI (la library de python)

2.6 6. Inférence

Le service qui fait tourner les modèles à haute vitesse.

Rôle du composant : Le moteur d'inférence représente l'infrastructure matérielle et logicielle (serveurs, puces spécialisées comme les GPU, TPU ou LPU) fournissant l'écrasante puissance de calcul nécessaire pour "animer" le Modèle de Langage. C'est l'essence même du système qui exécute en une fraction de seconde les milliards d'opérations mathématiques requises par le modèle IA pour lire la question de l'utilisateur et générer dynamiquement sa réponse mot par mot.

- **Groq API** : Le leader absolu pour la vitesse (LPUs).
- **SambaNova** : Un concurrent sérieux de Groq, optimisé pour les modèles Llama.
- **Cerebras** : Spécialisé dans le débit massif pour les très gros modèles.
- **Fireworks AI** : Excellent compromis entre vitesse constante et coût.
- **Together AI** : La plus grande bibliothèque de modèles open-source en API.

Choix retenu : Conflit entre Groq API et le local

2.7 7. Framework

Le chef d'orchestre qui lie tous les composants.

Rôle du composant : Pour aboutir à l'application logicielle finale, les différents outils mentionnés ci-dessus doivent collaborer dans un pipeline bien déterminé. L'orchestrateur (ou le *Framework*) sert d'architecture logicielle englobante. C'est ce bloc de code structuré (généralement en Python ou TypeScript) qui gère l'historique des conversations, formate les prompts envoyés au modèle, prend la question de l'utilisateur pour l'envoyer au modèle d'Embedding, requête la Base de données avec ce vecteur, récupère la réponse, et orchestre le passage de l'information finale au LLM.

- **LangChain** : L'écosystème le plus vaste et complet.
- **LlamaIndex** : Le meilleur pour tout ce qui touche à la gestion des données RAG.
- **Haystack** : Très apprécié pour les pipelines de production robustes.
- **LangGraph** : Pour créer des agents IA capables de boucles de réflexion.
- **PydanticAI** : Nouveau venu très populaire pour sa simplicité et sa validation de données.

Choix retenu : LangChain

3 Conclusion

La réussite d'un projet RAG repose fondamentalement sur la synergie entre ces sept piliers architecturaux, chaque composant ayant un rôle technique incontournable depuis l'extraction des données brutes jusqu'à la formulation de la réponse logique finale. La sélection de chaque module dans notre projet tiendra compte de nos exigences de performances (latence via Groq), des modèles de pointe retenus, et de la complexité visuelle des documents universitaires que nous aurons à indexer.